Appendix. Full List of Cause Categories

C1 Inconsistent assumptions (23). This is the top cause of core challenges. It represents the challenges caused by two components, or a component and its input, disagreeing on what a data structure contains. It has four subcategories: (i) *producer and consumer mismatch* (12): one component produces an IR, graph, or model that another component interprets differently, e.g., OPAL's three-address code generator assumed that a returned value is always held by the variable with ID 0, which instructions such as `dup` that rearrange the operand stack do not guarantee, so return statements referenced the wrong variable (OPAL-151, Figure 1a). Another such mismatch is between Soot's exceptional control-flow graph and Shimple's phi elimination (Soot-385). (ii) *synthetic entities not considered* (3): the framework creates a synthetic entity that some components do not account for, e.g., SVF builds a dummy entry point `svf.main` that calls the global constructors, but several analyses still hard-coded `main` as the entry function and missed the function-pointer assignments made during initialization (SVF-1818, Figure 1b). (iii) *one entity, two identities* (2): the same program entity is represented in two ways that are not reconciled, e.g., SVF kept the namespace in C++ class names when building the class hierarchy but removed it when resolving virtual calls (SVF-1369). (iv) *input assumed well-formed* (6): wrong assumptions about the input, such as that every class resolves, every call has a callee, or every constructor or destructor has at least one argument, e.g., SVF's class hierarchy builder looked up the callee of every call in a constructor and dereferenced it before checking for null, so an indirect call through a member-function pointer, which has no static callee, crashed the builder (SVF-204, Figure 1c).

Implication C1: Framework developers should make contracts between components explicit and check them, e.g., IR invariants after each pass.

C2 Unmodeled external behavior (16). This category represents the challenges caused by the analysis missing behavior that happens outside the analyzed code. It has four subcategories: (i) *runtime and lifecycle* (3): entry points, callbacks, initializers, and implicitly thrown exceptions, e.g., the global constructors of a C++ program run before `main` and are never called in the IR, so SVF's call graph missed their stores. This is fixed by calling them from the synthetic entry point `svf.main`, which takes them from LLVM's `llvm.global_ctors` list instead of matching a function-name pattern (SVF-760). (ii) *reflection, native, and dynamic code* (5): behavior that is not part of the analyzed code, e.g., Soot does not model the native method behind `Class.getDeclaredFields()`, so reflective field reads have empty points-to sets (Soot-2288, Figure 1d). (iii) *library behavior* (5): behavior that is in the libraries and not part of the analyzed code, e.g., has no summary, or falls back to a wrong default (SVF-195). (iv) *target-platform constraints* (3): behavior that is specific to the target platform, e.g., limits of the Dex format (Soot-565).

Implication C2: Framework developers should derive models of external behavior from the program and its platform instead of from fixed patterns, and should also report the language features that are left unmodeled and document what the analysis assumes about them.

C3 Toolchain and language evolution (9). This category captures the challenges caused by a change outside the framework that breaks it. It has three subcategories: (i) *compiler or IR version* (3): a new compiler or LLVM release changes the IR, e.g., an LLVM release that changes the bitcode (SVF-1774). (ii) *runtime, library, or tool version* (3): a new JDK, library, or build-tool version changes the input or breaks the build (WALA-1490). (iii) *new language features* (3): a new language construct appears in the input, e.g., since Java 9, `javac` compiles string concatenation into `invokedynamic` calls bootstrapped by a class that Soot did not load (Soot-1313). The first two are changes outside the framework, the third is a new language feature in the input.

Implication C3: Framework developers should decouple their front ends from specific compiler, runtime, and language versions, and test against several versions, including upcoming ones, in continuous integration, so that a new release is supported before users run into it.

C4 Low-level IR gap (13). This category captures the challenges caused by the analysis missing information that is present in the source code but not in the IR. It has three subcategories: (i) *source structure reconstruction* (3): the analysis misses structure that is not explicit in the IR, e.g., LLVM IR does not record C++ inheritance, so SVF infers it from constructor calls and mistook the initialization of a member object for a base class (SVF-1539). (ii) *IR shapes not handled* (5): the analysis misses instructions or IR patterns that the code does not handle, e.g., SVF's graph builder had no handler for LLVM's `va_arg` and `freeze` instructions, so their results had no defining edge and building the value-flow graph failed an assertion (SVF-440, Figure 1e). (iii) *source info lost or inconsistent* (5): a step that produces IR drops line numbers or variable names, or debug tables disagree, e.g., Soot's Dex front end attached no line numbers to the units of four kinds of instructions, which therefore reported line -1 (Soot-893).

Implication C4: Framework developers should add support for the missing IR shapes and source information, e.g., by adding handlers for new instructions or by adding line numbers and variable names to the IR.

C5 Class loading and scope (17). This category captures the challenges caused by the analysis not seeing the program it

```
-  case IRETURN.opcode =>
-    returnInstruction(OperandVar.IntReturnValue)
+  case IRETURN.opcode | ... | ARETURN.opcode =>
+    returnInstruction(stack.head)
```

(a) OPAL-151 (C1.1): returned value assumed to be variable 0

```
   bool isProgEntryFunction(const Function* fun) {
-    return fun && fun->getName() == "main";
+    const char* main_name =
+        Options::SVFMain() ? "svf.main" : "main";
+    return fun && fun->getName() == main_name;
```

(b) SVF-1818 (C1.2): synthetic entry point `svf.main` not considered

```
-  const Function* callee = getCallee(cs)->getLLVMFun();
-  if (callee == NULL)
-      return;
+  if (getCallee(cs) == NULL)
+      return;
+  const Function* callee = getCallee(cs)->getLLVMFun();
```

(c) SVF-204 (C1.4): null callee of an indirect call dereferenced

```
+} else if (subSignature.equals(
+    "java.lang.reflect.Field[] getDeclaredFields0(boolean)")) {
+  java_lang_Class_getDeclaredFields0(...);
+  return;
```

(d) Soot-2288 (C2.2): native `getDeclaredFields0` not modeled

```
+/// %tmp = va_arg i8* %ap2, i32
+void PAGBuilder::visitVAArgInst(VAArgInst &inst) {
+  NodeID dst = getValueNode(&inst);
+  NodeID src = getValueNode(inst.getPointerOperand());
+  addCopyEdge(src, dst);
+}
```

(e) SVF-440 (C4.2): LLVM's `va_arg` instruction not handled

```
   mpag.addInternalEdge(src, dest);
+if (opts.allocate_params()
+    && (method.isPublic() || method.isProtected())
+    && rhs instanceof ParameterRef && lhsType instanceof RefType) {
+  Node alloc = pag.makeAllocNode(rhs, AnySubType.v(lhsType), method);
+  mpag.addInternalEdge(alloc, src);
+}
```

(f) Soot-441 (C5.3): no objects for parameters of uncalled library methods

```
   // command line
+-p jb use-original-names:true
   // in code, before any class is loaded
+Options.v().setPhaseOption("jb", "use-original-names:true");
```

(g) Soot-1576 (C6.3): original variable names dropped unless enabled

```
   // Scene.java: class table
-Map<String, RefType> nameToClass = new HashMap<String, RefType>();
+Map<String, RefType> nameToClass = new ConcurrentHashMap<>();
   // HashChain.java: class chains
-public boolean add(E item) {
+public synchronized boolean add(E item) {
```

(h) Soot-1122 (C8.2): shared class table not synchronized

Fig. 1. Code fix and configuration examples for different causes.

TABLE I
CAUSES AND SUBCATEGORIES OF THE ANALYZED CHALLENGES PER GROUP.

	Core					
Cause	**Repr.**	**Alg.**	**NF**	**Usage**	**SE**	**Total**
C1 Inconsistent assumptions	16	6	0	0	1	**23**
C1.1 Producer and consumer mismatch	9	3	0	0	0	12
C1.2 Synthetic entities not considered	2	1	0	0	0	3
C1.3 One entity, two identities	0	2	0	0	0	2
C1.4 Input assumed well-formed	5	0	0	0	1	6
C2 Unmodeled external behavior	15	1	0	0	0	**16**
C2.1 Runtime and lifecycle	2	1	0	0	0	3
C2.2 Reflection, native, and dynamic code	5	0	0	0	0	5
C2.3 Library behavior	5	0	0	0	0	5
C2.4 Target-platform constraints	3	0	0	0	0	3
C3 Toolchain and language evolution	8	0	0	0	1	**9**
C3.1 Compiler or IR version	3	0	0	0	0	3
C3.2 Runtime, library, or tool version	2	0	0	0	1	3
C3.3 New language features	3	0	0	0	0	3
C4 Low-level IR gap	9	4	0	0	0	**13**
C4.1 Source structure reconstruction	1	2	0	0	0	3
C4.2 IR shapes not handled	3	2	0	0	0	5
C4.3 Source info lost or inconsistent	5	0	0	0	0	5
C5 Class loading and scope	9	1	4	3	0	**17**
C5.1 Incomplete classpath	6	0	0	2	0	8
C5.2 Load order and duplicates	1	0	4	0	0	5
C5.3 Analysis setup	2	1	0	1	0	4
C6 Deliberate limits	4	8	5	11	0	**28**
C6.1 Precision limits	1	5	0	3	0	9
C6.2 Scalability trade-offs	0	2	5	0	0	7
C6.3 Off by default or unfinished	3	1	0	1	0	5
C6.4 Outside the design scope	0	0	0	4	0	4
C6.5 Unreleased research prototype	0	0	0	3	0	3
C7 Local implementation errors	11	5	2	0	0	**18**
C7.1 Logic error	11	5	0	0	0	16
C7.2 Inefficiency or resource leak	0	0	2	0	0	2
C8 Ordering and concurrency	1	2	3	0	0	**6**
C8.1 Phase and analysis ordering	1	2	0	0	0	3
C8.2 Concurrency and iteration order	0	0	3	0	0	3
C9 Evolution and design debt	3	2	0	9	6	**20**
C9.1 Incomplete internal migration	2	0	0	1	0	3
C9.2 Design debt	1	2	0	3	3	9
C9.3 Hard-coded behavior, no extension point	0	0	0	5	0	5
C9.4 Evolution not managed for users	0	0	0	0	3	3
C10 Non-obvious semantics	3	1	0	17	0	**21**
C10.1 API semantics and defaults	2	1	0	8	0	11
C10.2 Artifacts of framework models	1	0	0	1	0	2
C10.3 Analysis semantics and IR encoding	0	0	0	5	0	5
C10.4 Hidden option requirements	0	0	0	3	0	3
C11 Internal state not exposed	0	0	0	9	0	**9**
C12 Project infrastructure	0	0	0	0	11	**11**
C12.1 Build assumptions and dependencies	0	0	0	0	2	2
C12.2 Fragile or incomplete tests	0	0	0	0	3	3
C12.3 Outdated documentation	0	0	0	0	3	3
C12.4 Governance and hosting	0	0	0	0	3	3
Undetermined	11	10	6	6	5	**38**
Total	90	40	20	55	24	**229**

Repr.: program representation, modeling, and transformation; Alg.: analysis algorithms and correctness; NF: non-functional analysis challenges; SE: general software engineering.

is meant to analyze. It has three subcategories: (i) *incomplete classpath* (8): classes are missing from the analysis scope, or exclusions remove them, often because the user left the scope incomplete, e.g., Soot stopped because library classes referenced by an APK were not on its classpath, and the maintainers pointed to an option that creates phantom classes for them (Soot-76). (ii) *load order and duplicates* (5): the program depends on which of several definitions of a class is used, in which order classes are loaded, and by which class loader, e.g., `eclipse.jar` bundles conflicting class files for the same class, and OPAL keeps the first one it parses, so its class hierarchy and call graph changed with the order in which the nested jars were parsed (OPAL-198). (iii) *analysis setup* (4): the entry points, the closed-world assumption, or the platform inputs that the user configures do not match the program, including inputs passed through the wrong option, e.g., Spark assumes that every call into a method is visible, so when Soot analyzed a library, the parameters of public methods that no analyzed code calls had empty points-to sets,

and virtual calls on them had no call-graph edges (Soot-441, Figure 1f).

Implication C5: Framework developers should document guidance on how to set up the analysis scope, i.e., which classes must be on the classpath, which entry points and world assumptions the analysis uses, and which option takes which input.

C6 Deliberate limits (28). This category captures the challenges caused by a design decision of the analysis or the framework, not by a defect. It has five subcategories: (i) *precision limits* (9): the analysis is context-, field-, array-, path-, or thread-insensitive or performs only weak updates by design, e.g., SVF's mod-ref analysis is context-insensitive, so a value written by one call of a function flowed to another call of it, and fixing this would need a context-sensitive pointer analysis (SVF-10). (ii) *scalability trade-offs* (7): the analysis is designed in a way that does not scale to large programs, such as dense propagation or whole-program analysis, e.g., Spark builds the whole call graph up front because resolving interface calls precisely needs the whole program, so even a client that queries a few methods pays for loading and converting all of it (Soot-166). (iii) *off by default or unfinished* (5): a capability is disabled by default or was never finished, e.g., Soot drops the original variable names unless an option is set before classes are loaded, so a user's decompiled code showed Soot's generated names instead (Soot-1576, Figure 1g). (iv) *outside the design scope* (4): the user expects something the framework is not designed for, e.g., a user asked to run PhASAR on source-level Clang CFGs, but PhASAR is designed to analyze LLVM IR only (PhASAR-482). (v) *unreleased research prototype* (3): a technique was published but never released, e.g., a user asked for SnuggleBug, a WALA-based analysis described in a paper, but it was never made open source (WALA-1034).

Implication C6: Beyond documenting the deliberate limits of each analysis, framework developers should state what the framework does not support, such as input languages, IRs, or published techniques that were never released, so that users can tell a limit from a defect before reporting it.

C7 Local implementation errors (18). This category captures the challenges caused by a defect within one algorithm. It has two subcategories: (i) *logic error* (16): a wrong argument, operator, value, or traversal inside one algorithm, e.g., WALA's query for the least common superclass of `a` and `b` compared the ancestors of `a` with the superclasses of `b` but not with `b` itself, so it missed the case in which `b` is a superclass of `a` (WALA-53). (ii) *inefficiency or resource leak* (2): an inefficient data structure or algorithm, or memory that is never released, e.g., when SVF's versioned flow-sensitive analysis added a call-graph edge, it re-propagated points-to sets to every dependent version instead of only the newly connected one, which slowed it down on programs where one call-graph update adds over 200k edges (SVF-495).

Implication C7: Framework developers should test each algorithm on the corner cases of its input, e.g., with unit tests per language construct or instruction and with reference results, since an error in one case of an algorithm only shows up when a program exercises that case.

C8 Ordering and concurrency (6). This category captures the challenges caused by the result depending on the order of execution. It has two subcategories: (i) *phase and analysis ordering* (3): a phase or analysis uses information that another one produces before that information is available or final, e.g., WALA creates the classes modeled by XML summaries only after the class hierarchy is built, so a class that extends a modeled class could not find its superclass and fell back to the root class (WALA-1958). (ii) *concurrency and iteration order* (3): the result depends on thread scheduling or on the iteration order of collections, e.g., Soot converts method bodies in parallel but accessed its shared class table without synchronization, so its tests failed in about one of five CI runs (Soot-1122, Figure 1h).

Implication C8: Framework developers should make the dependencies between phases and analyses explicit and check them, protect state that is shared between threads, and use collections with a deterministic iteration order.

C9 Evolution and design debt (20). This category captures the challenges caused by the framework's own structure. It has four subcategories: (i) *incomplete internal migration* (3): after an internal format or API changes, some code paths are left behind, e.g., SVF's specification of external functions gained type signatures in release 2.6, but the helper that adds entries at run time was not updated, so the entries it created no longer matched the format (SVF-1083). (ii) *design debt* (9): an internal or API structure hinders use, maintenance, or extension, e.g., PhASAR required every edge function to implement `compose` and `join`, even in analyses that already defined these operations in one central place, so developers had to write placeholder methods that only raise an error (PhASAR-736). (iii) *hard-coded behavior with no extension point* (5): a fixed choice that users cannot replace, e.g., Soot printed all diagnostic output directly to the console, so applications that use Soot as a library could neither silence nor redirect it (Soot-407). (iv) *evolution not managed for users* (3): a refactoring removes a supported path, or support for a new version is not released, e.g., a refactoring of SVF's module abstraction removed the LLVM pass interface of its pointer analysis, so users who ran SVF through LLVM's `opt` could no longer invoke it (SVF-289).

Implication C9: Framework developers should provide extension points instead of hard-coded choices, carry internal format and API changes through all code paths, and announce removed or changed features to users together with a migration path.

C10 Non-obvious semantics (21). This category captures the challenges caused by the framework behaving as intended, but the user did not know the behavior. It has four subcategories: (i) *API semantics and defaults* (11): an API, configuration format, or feature expects or does more than its signature and documentation say, e.g., a WALA method expects a bytecode index, but a user passed an SSA instruction index, so the lookup of the instruction at a source line never matched (WALA-643). (ii) *artifacts of framework models* (2): the framework's own models produce surprising results, e.g., WALA's synthetic

root method calls an entry point only if it can instantiate every parameter type, so entry points with a parameter type outside the analysis scope silently drop out of the call graph (WALA-1067). (iii) *analysis semantics and IR encoding* (5): the user misunderstands how an analysis resolves targets or how the IR encodes a construct, e.g., Spark derives the targets of a virtual call from the objects its receiver may point to, so a receiver that is always `null` points to no object, and the call got no call-graph edge (Soot-190). (iv) *hidden option requirements* (3): an option has implicit preconditions or side effects that are not documented, e.g., with output options set, Soot frees method bodies once it writes them out, so a later analysis found them gone (Soot-883).

Implication C10: Framework developers should document what their APIs, options, analyses, and IR constructs expect and compute, including their preconditions and side effects, and turn answers to recurring questions into documentation.

C11 Internal state not exposed (9). This category captures the challenges caused by the framework holding information that users need, such as internal state, diagnostics, or results, but offering no way to inspect, visualize, or report it. It has no subcategories. For example, PhASAR stored its edge and jump functions in solver caches with no interface to inspect them, so users could not debug or tune their analyses (PhASAR-697).

Implication C11: Framework developers should offer interfaces to inspect, dump, and visualize intermediate results, such as solver state, models, and graphs, so that users can debug and tune their analyses without changing the framework.

C12 Project infrastructure (11). This category captures the challenges caused by how the project is built, tested, documented, or hosted, not by the framework code. It has four subcategories: (i) *build assumptions and dependencies* (2): build scripts assume a particular environment or omit a dependency (SVF-734). (ii) *fragile or incomplete tests* (3): tests encode exact outputs, miss a class of defects, or use an outdated configuration, e.g., OPAL's integration tests fixed the exact set of reachable methods, so they failed when call-graph improvements made more methods reachable (OPAL-238). (iii) *outdated documentation* (3), e.g., links broken by a website move (Soot-315). (iv) *governance and hosting* (3): legal or ownership questions, or infrastructure the project does not control (SVF-20).

Implication C12: Framework developers should declare all build dependencies explicitly, write tests that tolerate intended improvements of the results, and check documentation and its links in continuous integration.

Undetermined (38). For these cases, neither the discussion nor the code change establishes a cause, e.g., because the issue received no response, could not be reproduced, was never investigated, or is a pull request that states no problem.